Week 5 - Mini End-to-End Data Warehouse Project

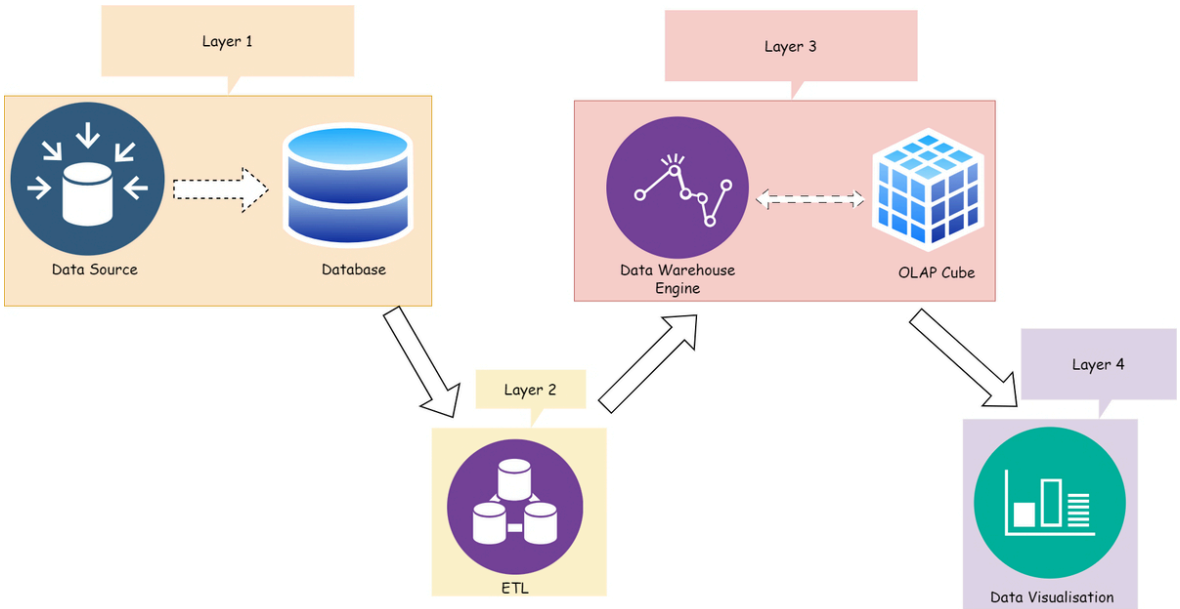
Copy

This lab contributes **5%** to your final grade.

In the Week 2 lab, we have already loaded the dataset to PostgreSQL. In Week 3, we practised ETL with databases. In this Lab, we will (**Layers 3 and 4 in the OLAP Architecture below**):

1. go through the process of designing a new data warehouse using Power BI or Tableau
2. build a cube for drill-down and roll-up analysis in Power Bi or Tableau
3. visualize the data using Tableau or Power BI

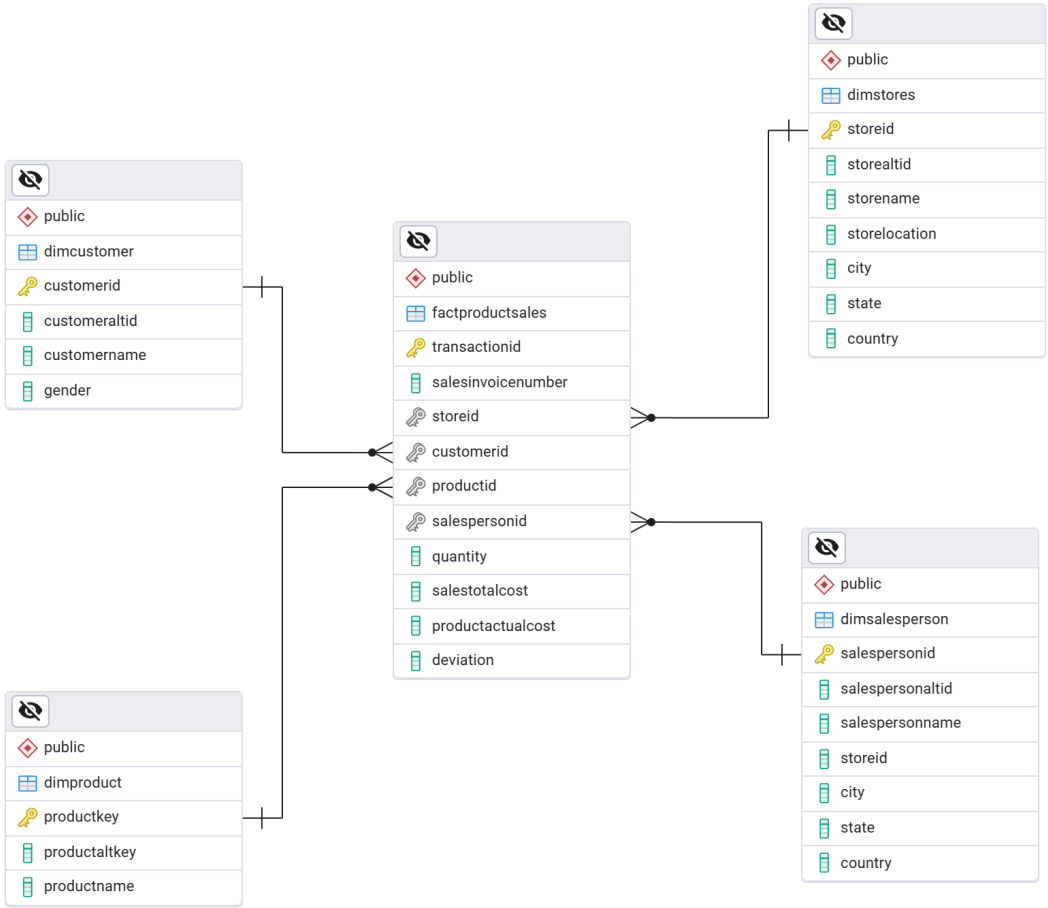
Part 1: OLAP Architecture



OLAP Architecture

Part 2 : Business Requirement

Woolworths have different shopping centers in Perth, where daily sales take place for various products. Higher management is facing an issue while decision making due to non-availability of integrated data they can't do study on their data as per their requirement. So they need to design a system which can help them quickly in decision making and provide Return on Investment (ROI). Let us start designing of the data warehouse, we need to follow a few steps before we start our data warehouse design. First, try to identify the dimension table, fact tables and the primary key, foreign key relationships. One of the possible designs is shown in the following figure. You need to identify the Fact table and the dimension tables with their relationships from the diagram below.



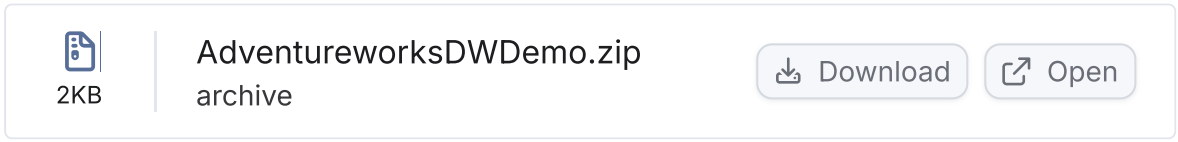
What is this schema?

Part 3: Data Warehouse Creation using PostgreSQL

> If you are using the Docker solution, **please skip this sections 3.0, 3.1 and 3.2**. We have already completed this part with Docker.

3.0 Dataset for Week 5 Lab

- If you use the local solution, please download the dataset.



Week 5 Lab Dataset

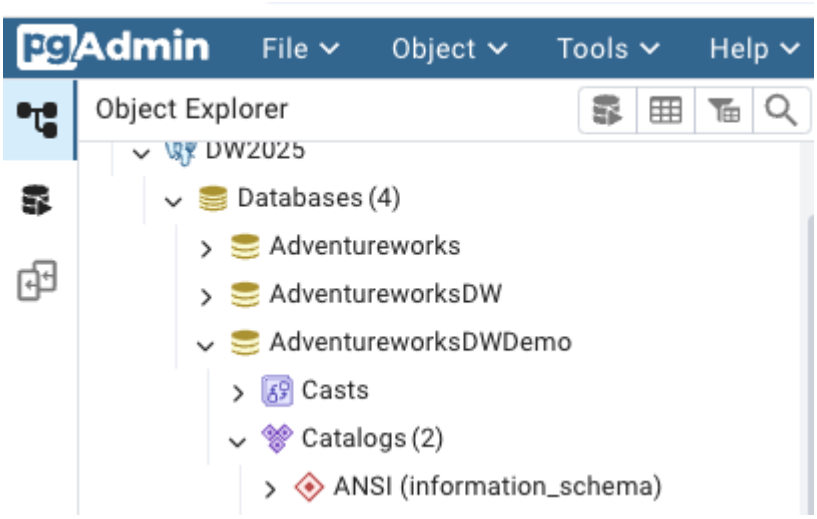
- If you use the Docker solution, please use the `AdventureworksDWDemo` database.

3.1 Create a PostgreSQL Database

As we mentioned in Lab4, there are two options for creating a database.

- **Option 1: Using the pgAdmin 4 GUI tool.**
  - Right-click the **server name** to expand the list.
  - Select "**Create**" then "**Database...**".
  - Enter `AdventureworksDWDemo` as the database name.
- **Option 2: Using the pgAdmin 4 query tool**
  - Click '**Query Tool Workspace**' to open the PostgreSQL editor.
  - Select an existing server from the list, then click '**Connect & Open Query Tool**'. Leave all other parameters at their default values.

Finally, you can check the **Object Explore** panel to find the created database.



3.2 Create Database Tables and Insert Data

The next step is to create dimension tables and fact table(s). You have two options for setting up your database tables. Option 1 is to create tables directly in PostgreSQL and **insert data row by row**. Option 2 is to create tables and then **import data from structured CSV files**.

Option 1: Create Tables Directly in PostgreSQL and Insert Data Row By Row

Step 1: Manually define and create each table within your PostgreSQL environment using SQL commands, which involves defining the structure of each table using SQL commands and then inserting data into those tables. Those are some examples for create fact table and dimension tables.

```
-- Create dimension table for customers
CREATE TABLE dimcustomer (
    customerid INTEGER PRIMARY KEY,
    customeraltid VARCHAR(10),
    customername VARCHAR(50),
    gender VARCHAR(2)
);

-- Create dimension table for products
CREATE TABLE dimproduct (
    productkey INTEGER PRIMARY KEY,
    productaltkey VARCHAR(10),
    productname VARCHAR(100)
);

-- Create dimension table for stores
CREATE TABLE dimstores (
    storeid INTEGER PRIMARY KEY,
    storealtid VARCHAR(10),
    storename VARCHAR(100),
    storelocation VARCHAR(100),
    city VARCHAR(100),
    state VARCHAR(100),
    country VARCHAR(100)
);

-- Create dimension table for salespersons
CREATE TABLE dimsalesperson (
    salespersonid INTEGER PRIMARY KEY,
    salespersonaltid VARCHAR(10),
    salespersonname VARCHAR(100),
    storeid INTEGER,
    city VARCHAR(100),
    state VARCHAR(100),
    country VARCHAR(100)
);

-- Create fact table for product sales
CREATE TABLE factproductsales (
    transactionid BIGINT PRIMARY KEY,
    salesinvoicenumber INTEGER,
    storeid INTEGER,
    customerid INTEGER,
    productid INTEGER,
    salespersonid INTEGER,
    quantity DOUBLE PRECISION,
    salestotalcost NUMERIC,
```

```
    productactualcost NUMERIC,
    deviation DOUBLE PRECISION,
    FOREIGN KEY (storeid) REFERENCES dimstores(storeid),
    FOREIGN KEY (customerid) REFERENCES dimcustomer(customerid),
    FOREIGN KEY (productid) REFERENCES dimproduct(productkey),
    FOREIGN KEY (salespersonid) REFERENCES
dimsalesperson(salespersonid)
);
```

Step2 : After creating the table, you can insert data into it. Here's an example of inserting a single record into the `dimcustomer` table:

```
INSERT INTO dimcustomer (customeraltid, customername, gender)
VALUES ('CT123', 'John Doe', 'M');
```

There is no need to insert a value for `customerid` when using the `SERIAL` type in PostgreSQL.

Option 2: Create Tables Directly in PostgreSQL and Upload Structured CSV Files

Import data from pre-structured CSV files directly into PostgreSQL. This can be faster if the CSV files are properly formatted with headers that match the intended table schema.

Step 1: Follow the same steps as in Option 1 to create a table. Make sure the table schema matches the structure of the CSV file you want to import.

Step 2: The `COPY` command is used when you have access to the server's file system where PostgreSQL is running. The syntax is:

```
COPY dimcustomer FROM '/path/to/DimCustomer.csv' WITH (FORMAT
csv, HEADER false)
```

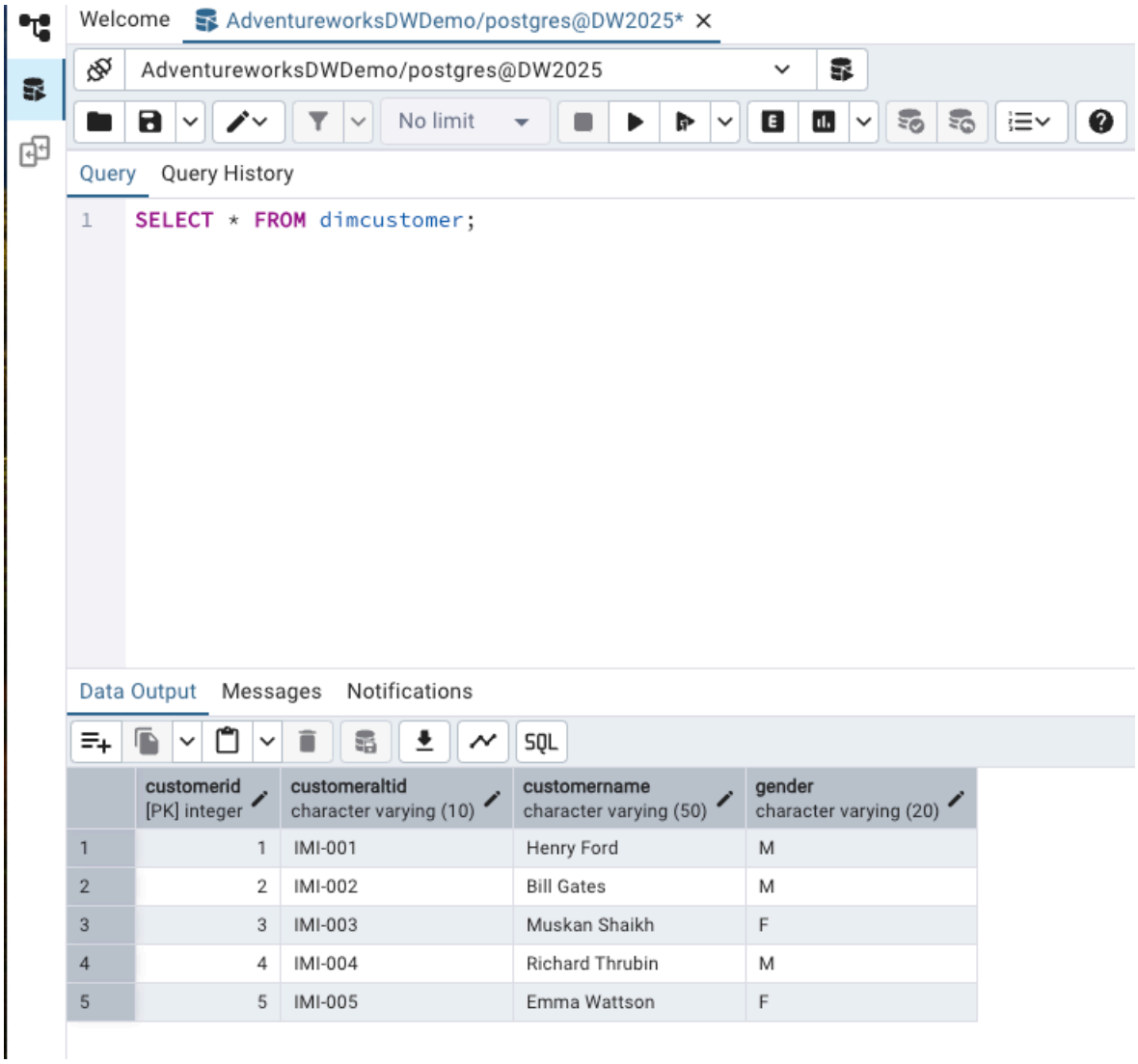
or you can use \copy Command in psql (does not require superuser privileges)

```
\copy dimcustomer FROM '/path/to/DimCustomer.csv' WITH (FORMAT
csv, HEADER false);
```

Please Note:

1. replace your table name with `dimcustomer`
2. ***File names are case-sensitive.***
3. replace `'/path/to/DimCustomer.csv'` or `'/path/to/Dimcustomer.csv'` with the actual path to your CSV file to bulk insert data.
4. For example, `\copy dimcustomer FROM '/tmp/DimCustomer.csv' WITH (FORMAT csv, HEADER false);`

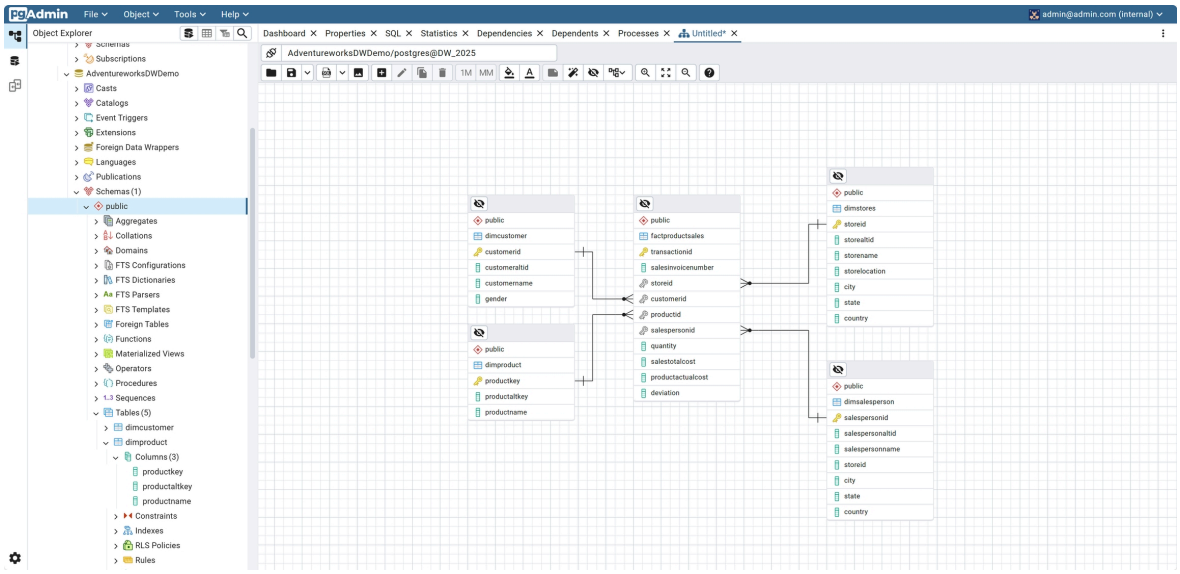
After importing the data, it is crucial to verify if the data has been imported correctly. Execute some SELECT queries to check if the data looks correct.



> If you are not familiar with this step, please refer to Week 4 lab: 1.1.

3.3 Create a schema

We can generate the schema for the fact and dimension tables using pgAdmin 4 GUI. Below is the expected database schema. You can modify both the table details and their layout as needed. (use ***ERD For Database*** to view this diagram)

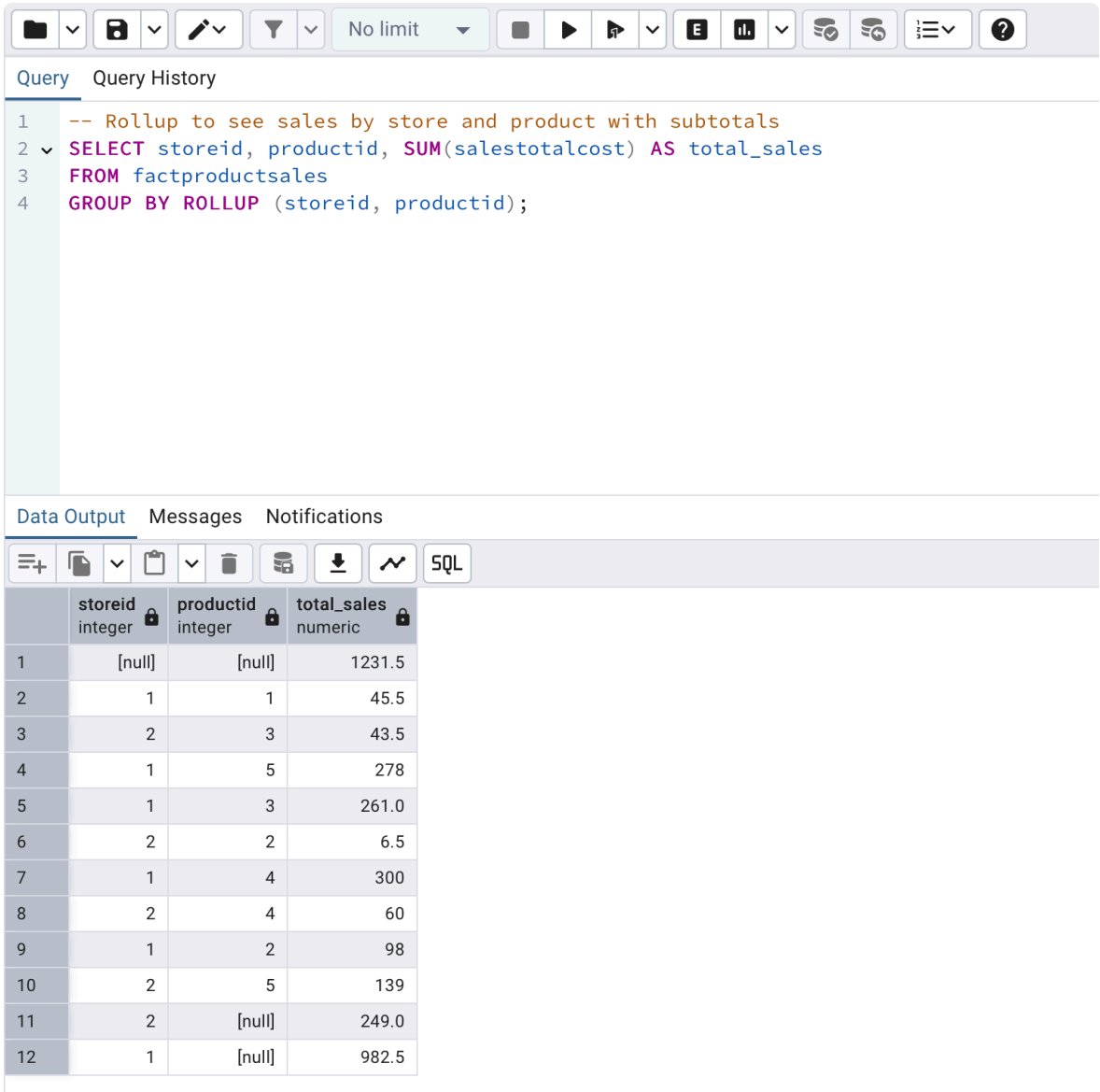


Part 4: Use PostgreSQL Queries for Rollup and Cube Aggregation

In PostgreSQL, the `ROLLUP` and `CUBE` functions are used in the `GROUP BY` clause to produce aggregated results that include subtotals and grand totals. These functions are particularly useful in generating reports that require multi-level aggregation across several dimensions, such as sales data analytics. Here's some examples of both queries:

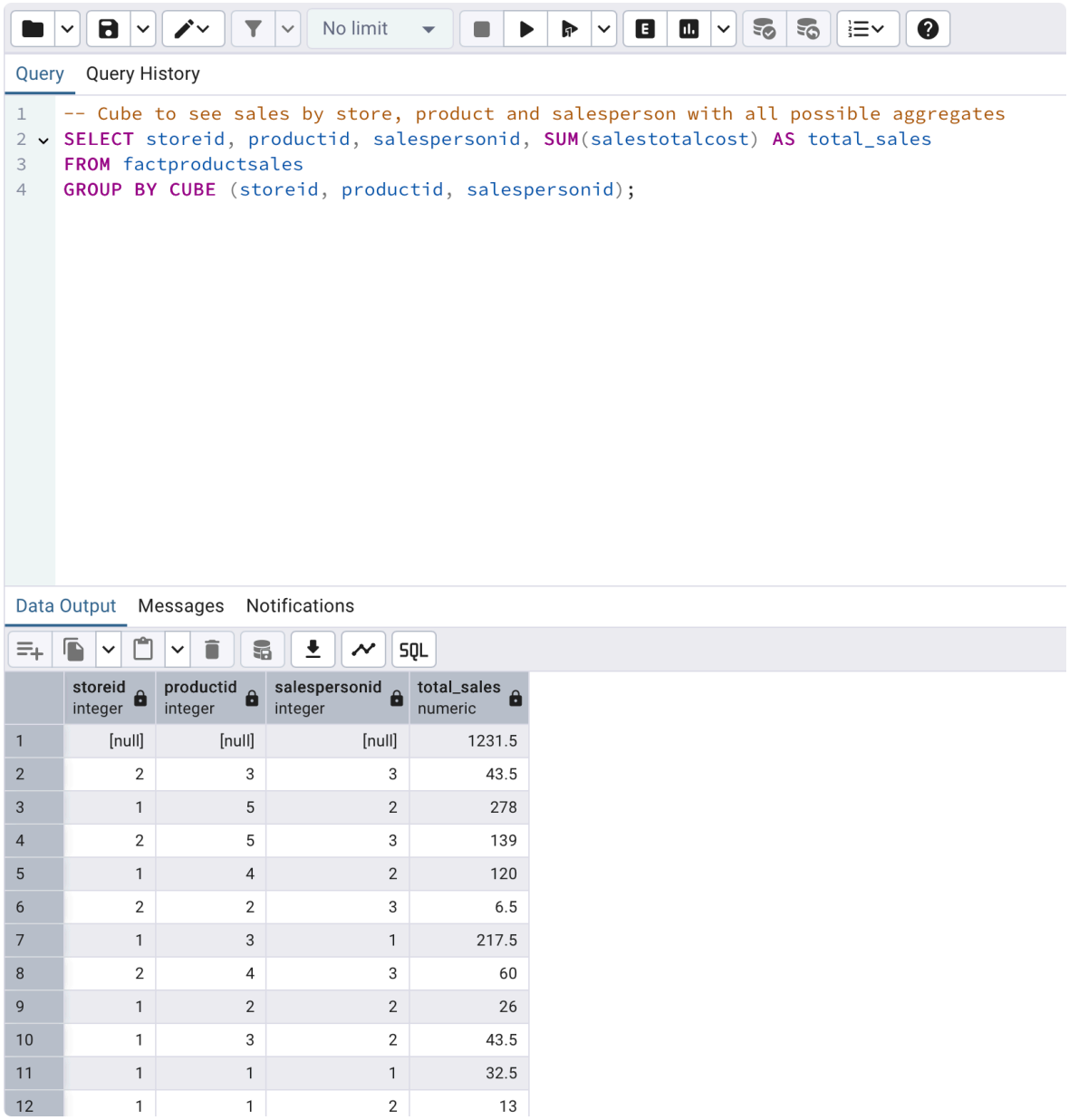
```
-- Rollup to see sales by store and product with subtotals
SELECT storeid, productid, SUM(salestotalcost) AS total_sales
FROM factproductsales
GROUP BY ROLLUP (storeid, productid);
```

```
-- Cube to see sales by store, product and salesperson with all
possible aggregates
SELECT storeid, productid, salespersonid, SUM(salestotalcost) AS
total_sales
FROM factproductsales
GROUP BY CUBE (storeid, productid, salespersonid);
```



	storeid integer	productid integer	total_sales numeric
1	[null]	[null]	1231.5
2	1	1	45.5
3	2	3	43.5
4	1	5	278
5	1	3	261.0
6	2	2	6.5
7	1	4	300
8	2	4	60
9	1	2	98
10	2	5	139
11	2	[null]	249.0
12	1	[null]	982.5

Sales data summary using SQL ROLLUP with store/product grouping and totals.



Query Query History

```
1  -- Cube to see sales by store, product and salesperson with all possible aggregates
2  SELECT storeid, productid, salespersonid, SUM(salestotalcost) AS total_sales
3  FROM factproductsales
4  GROUP BY CUBE (storeid, productid, salespersonid);
```

Data Output Messages Notifications

	storeid integer	productid integer	salespersonid integer	total_sales numeric
1	[null]	[null]	[null]	1231.5
2	2	3	3	43.5
3	1	5	2	278
4	2	5	3	139
5	1	4	2	120
6	2	2	3	6.5
7	1	3	1	217.5
8	2	4	3	60
9	1	2	2	26
10	1	3	2	43.5
11	1	1	1	32.5
12	1	1	2	13

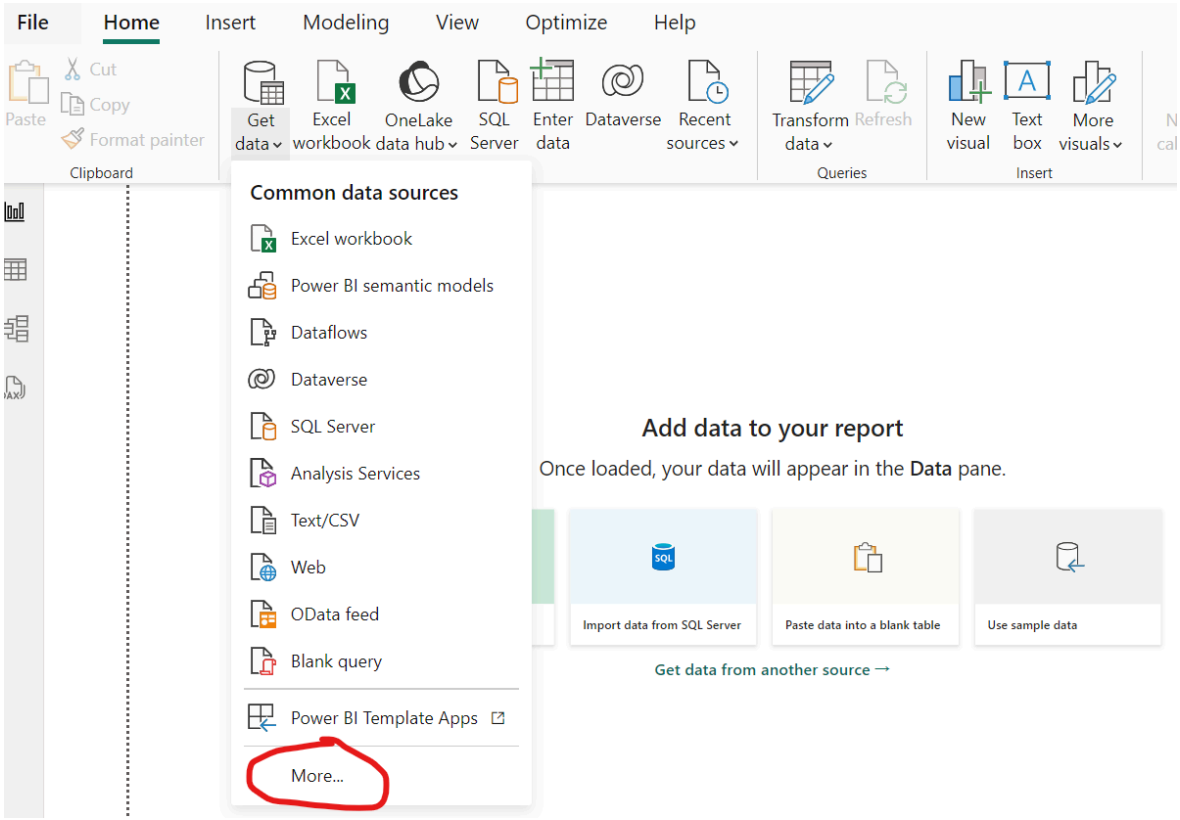
Sales data analysis using SQL CUBE function.

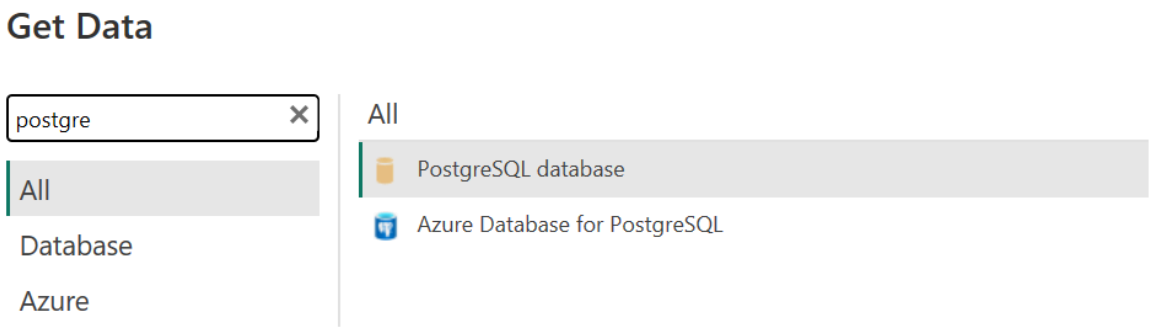
Part 5: Visualising the Multidimensional Solution using Power BI/Tableau

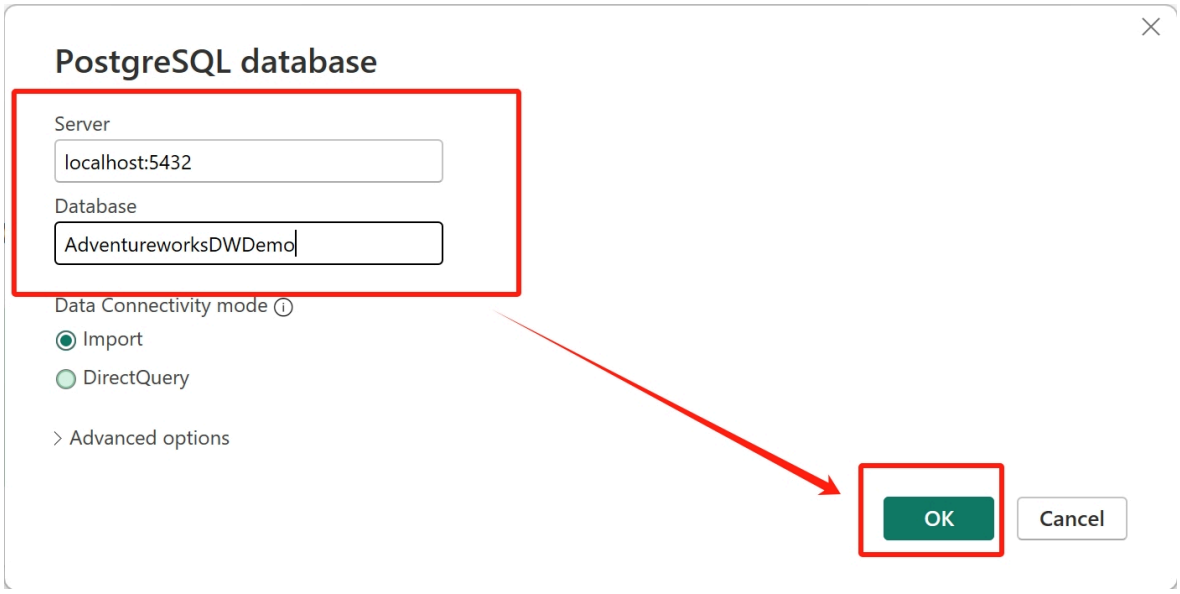
5.1 Perform Visualization in Power BI

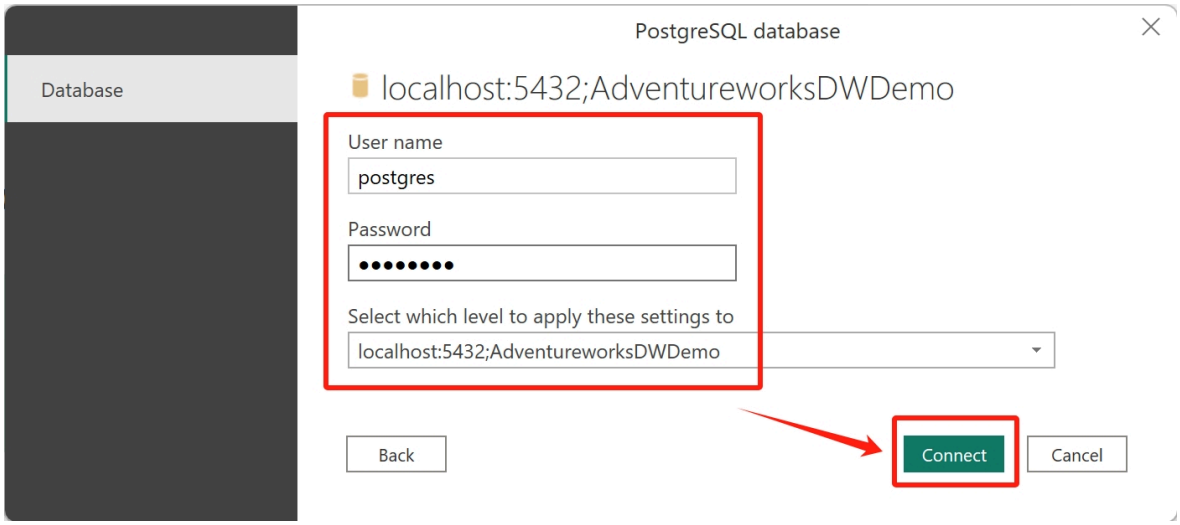
Step 1 : Connect to PostgreSQL Database:

Open Power BI. Select '**Get Data**' and choose '**PostgreSQL database**'. Enter the credentials and the server details to connect.





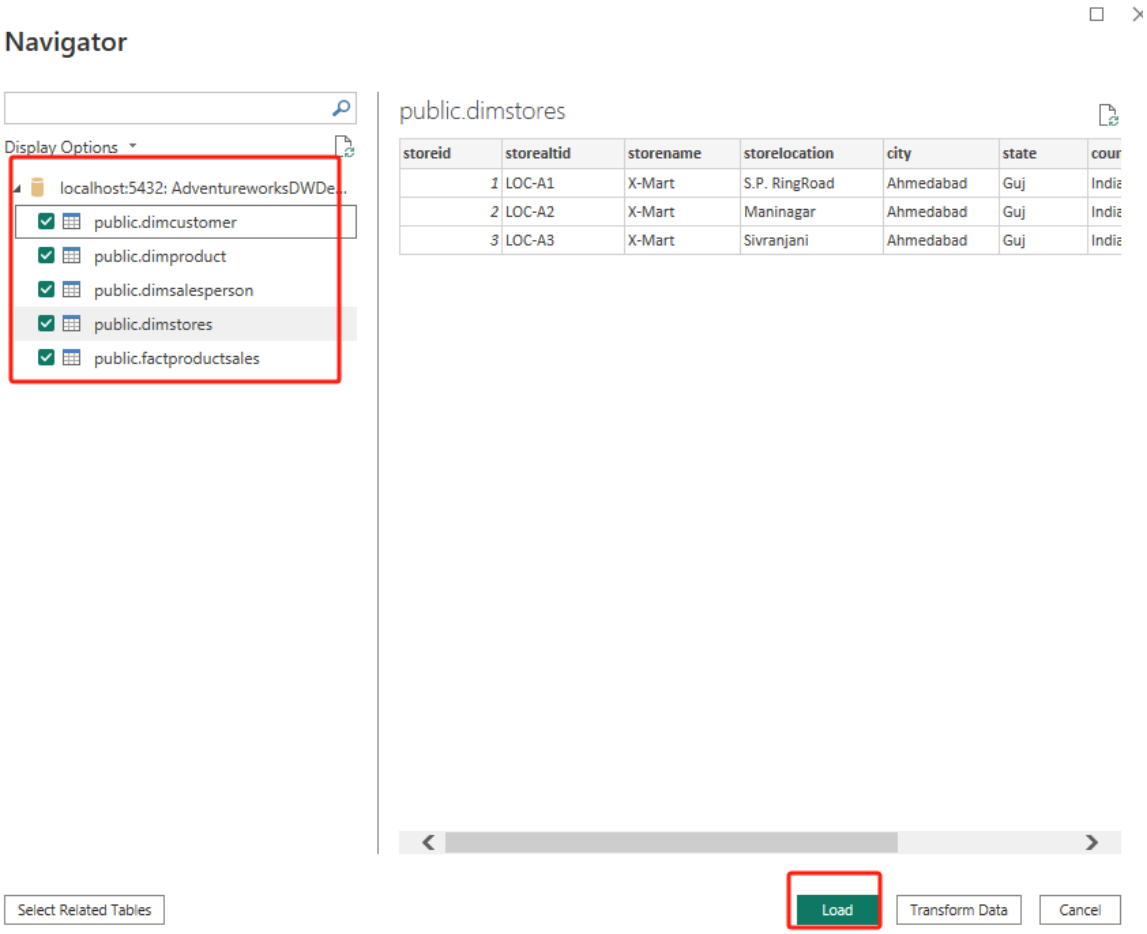




For more details, please refer to Week 4 Lab: Task 2.

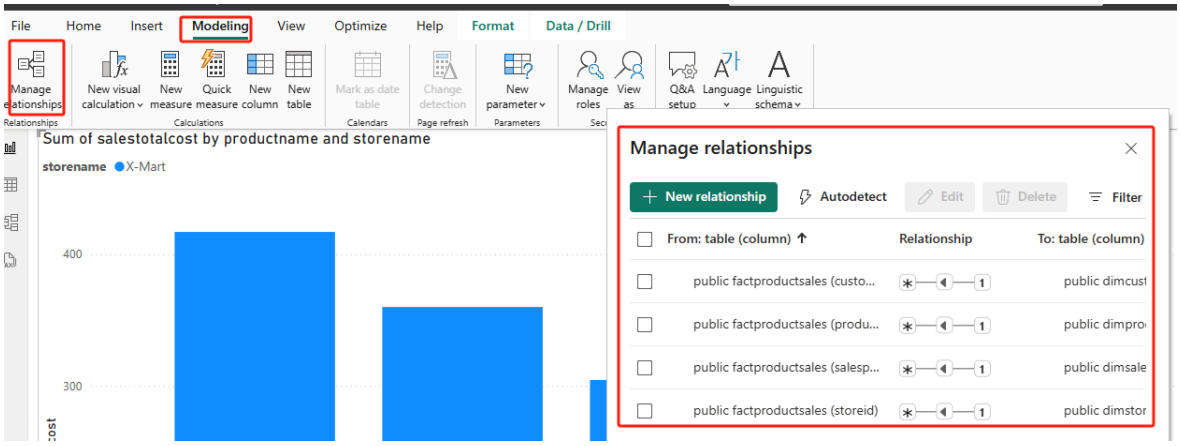
Step 2 : Load Data:

- Select the tables you've created and load them into Power BI.



Step 3 : Create Relationships:

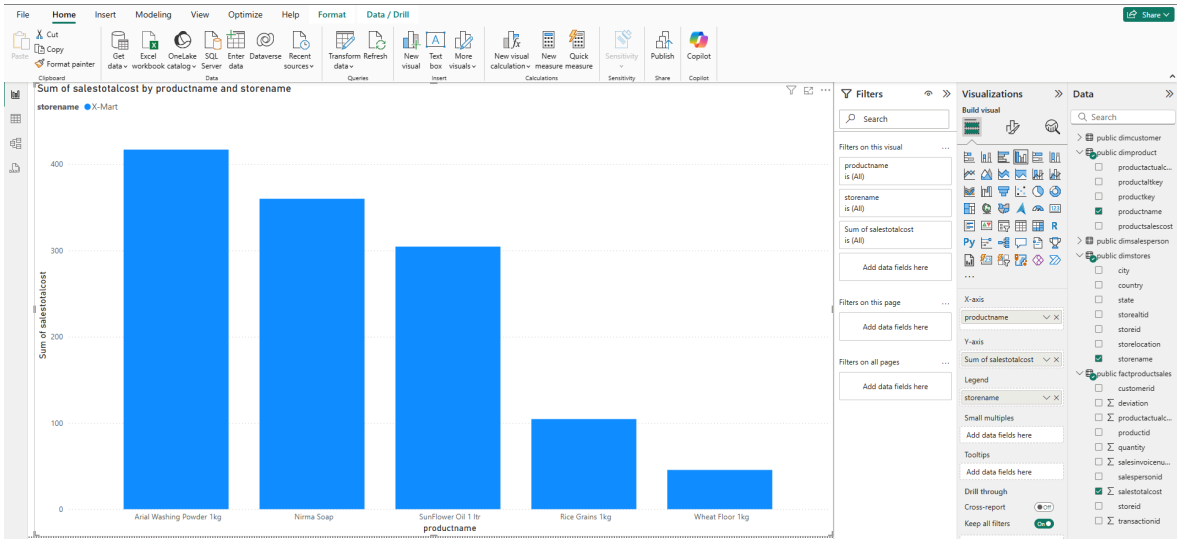
- Go to the 'Modeling' view in Power BI.
- Ensure all relationships are correctly set up between the fact and dimension tables as per your schema.



Step 4 : Create Visualizations:

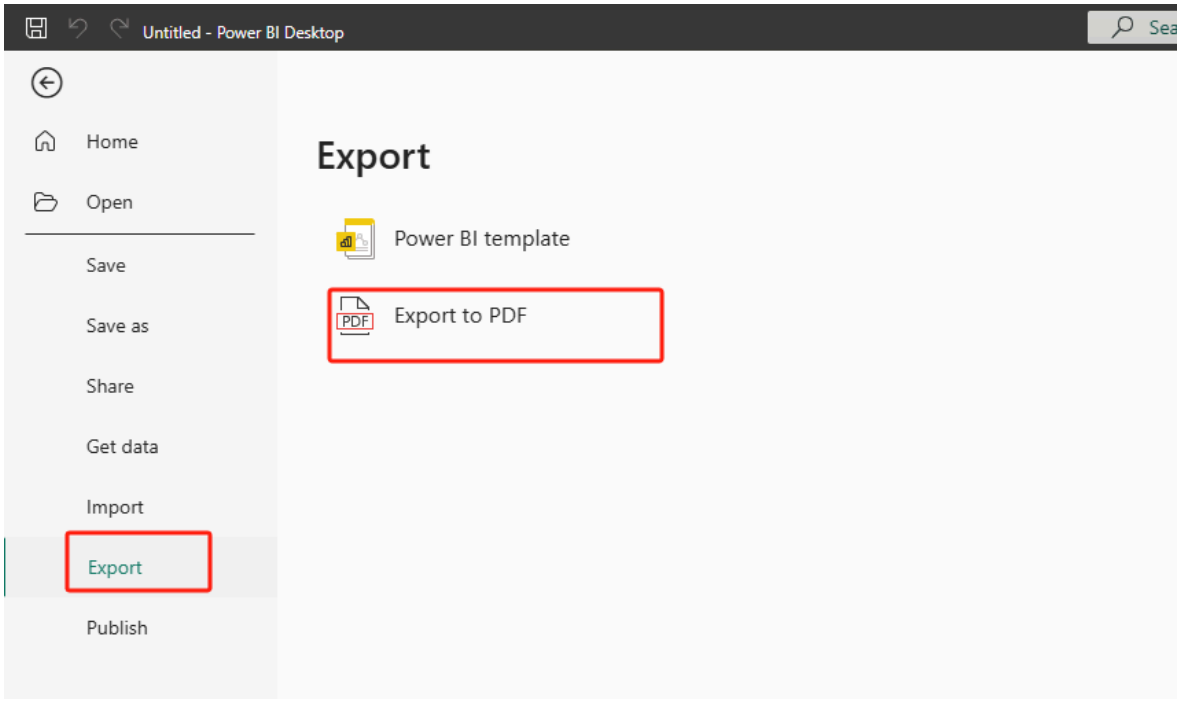
- Drag dimensions (e.g., `productmane`) and measures (e.g., `productsalestotalcost`) to the X and Y axes of a chart.

- Choose appropriate chart types, e.g., bar charts for sales data, pie charts for demographic distributions.



Step 5 : Publish and Share

- Publish the report to the Power BI service.
- Share it with stakeholders for interactive access.
- In the Power BI service, select **Export** > **Export to PDF** from the menu bar.

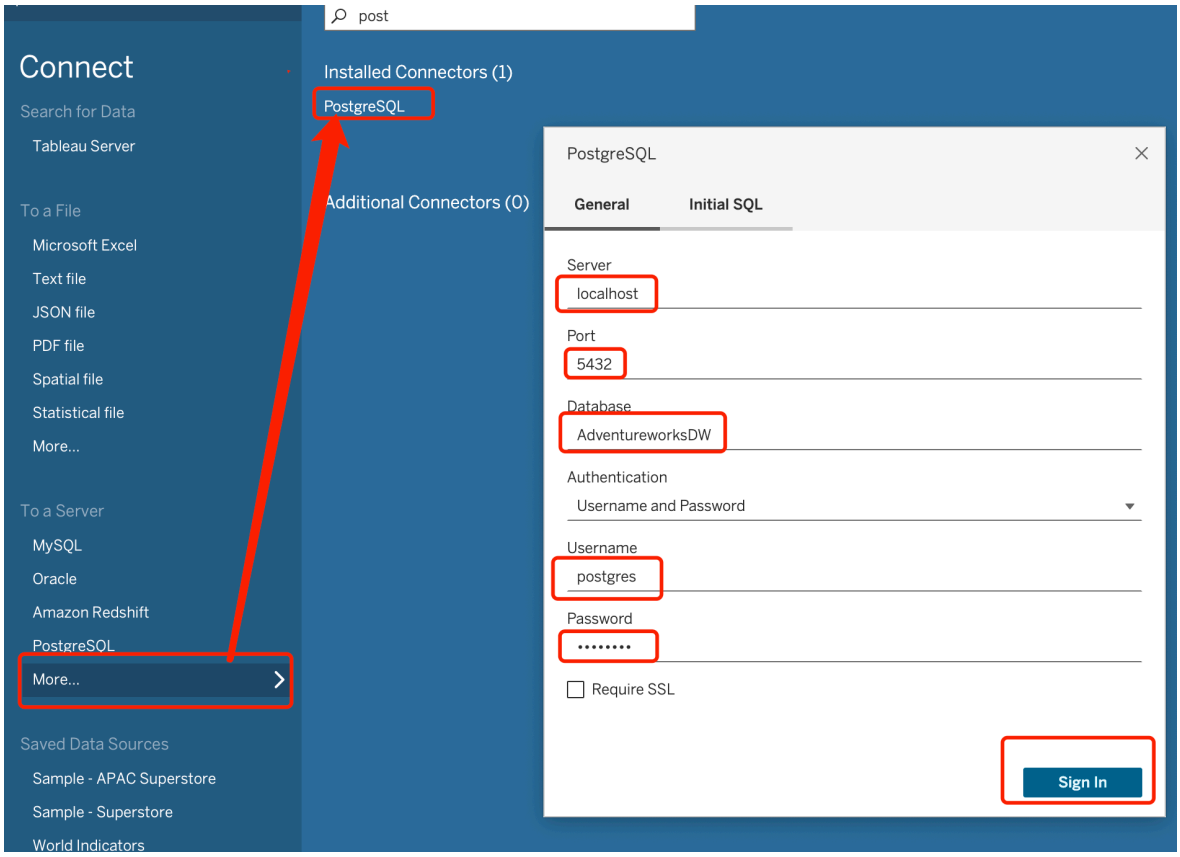


5.2 Perform Visualization in Tableau

Step 1: Connect to PostgreSQL Database

Start Tableau and go to the start page. Under the "**Connect**" section on the left, choose "**More...**" under "To a Server" and then select "**PostgreSQL**". Enter the details of your PostgreSQL database, and click "Sign In" to establish the connection.

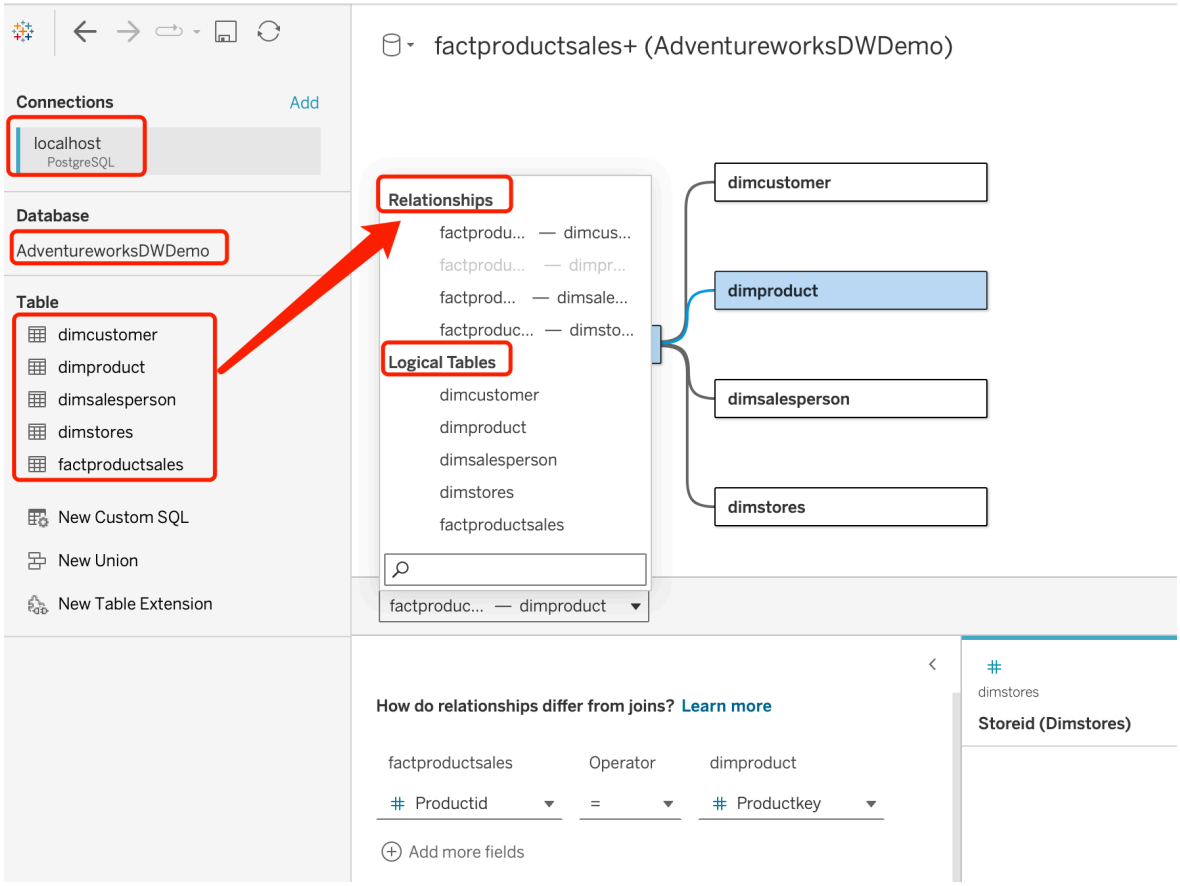
> For more details, you can refer to Week 4 Lab: 2.3.

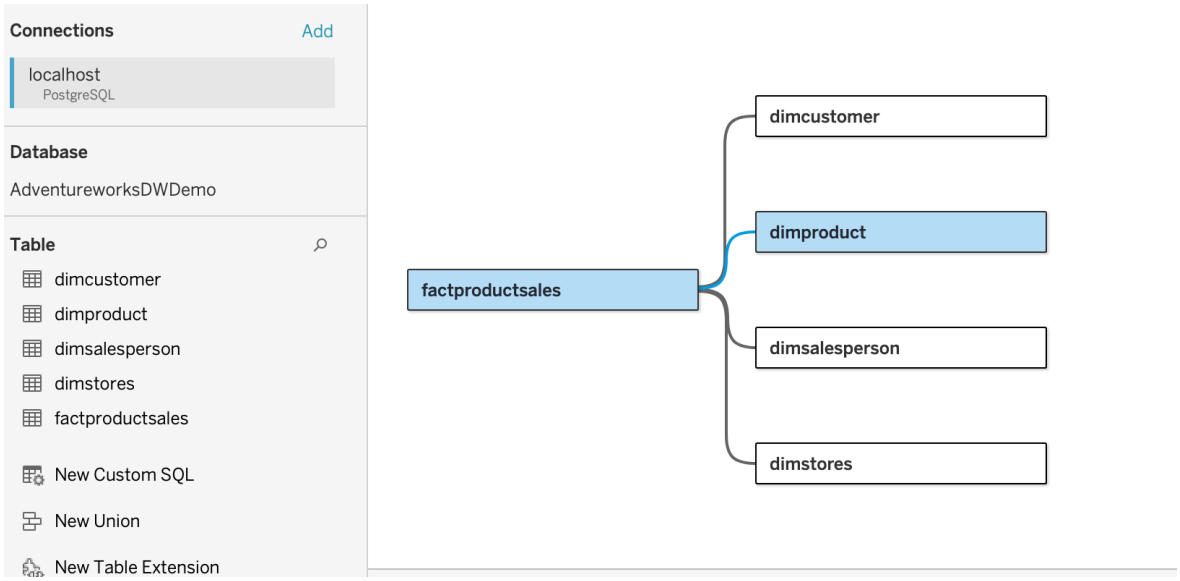


Step 2: Import Data

- After connecting, Tableau will display the list of tables in the database.
- Drag the tables (factproductsales, dimcustomer, dimproduct, dimsalesperson, dimstores) into the join area.

- Set up the joins according to the foreign keys defined in your database schema. For instance, join `factproductsales` to `dimstores` on `storeid`, and so on.
- Once the joins are configured, click on "Sheet 1" to start creating visualizations.

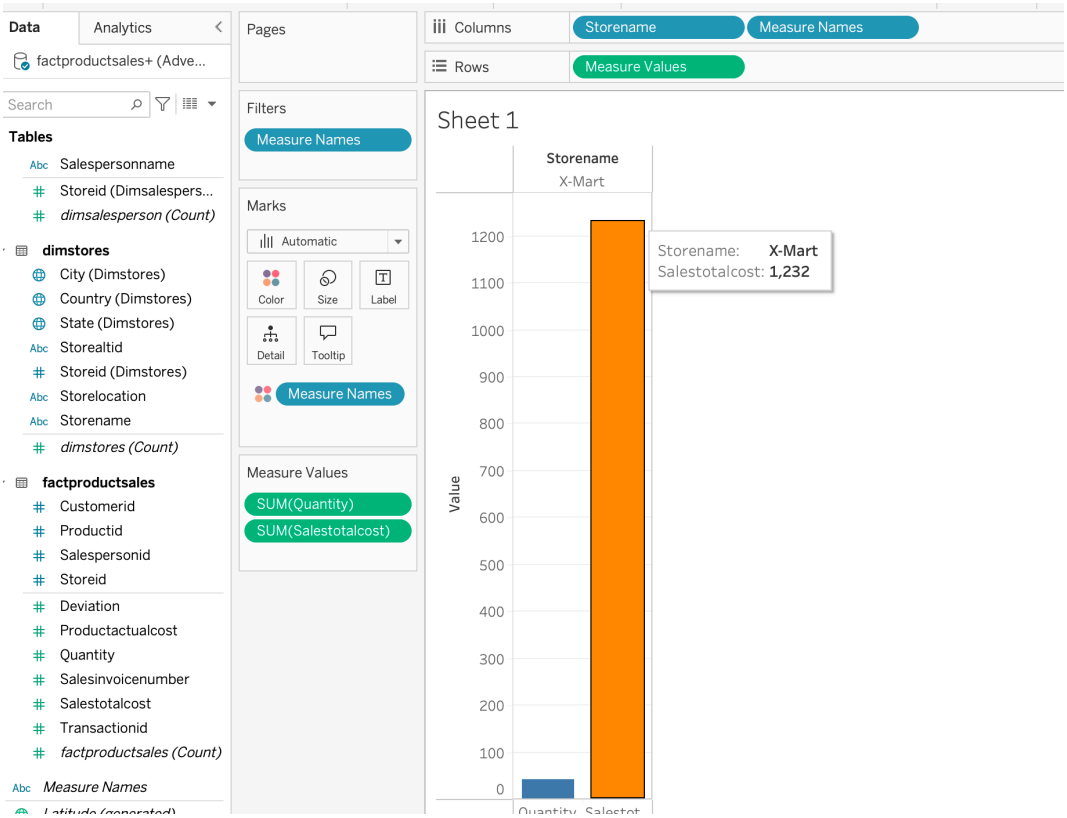




Step 3: Create Visualizations

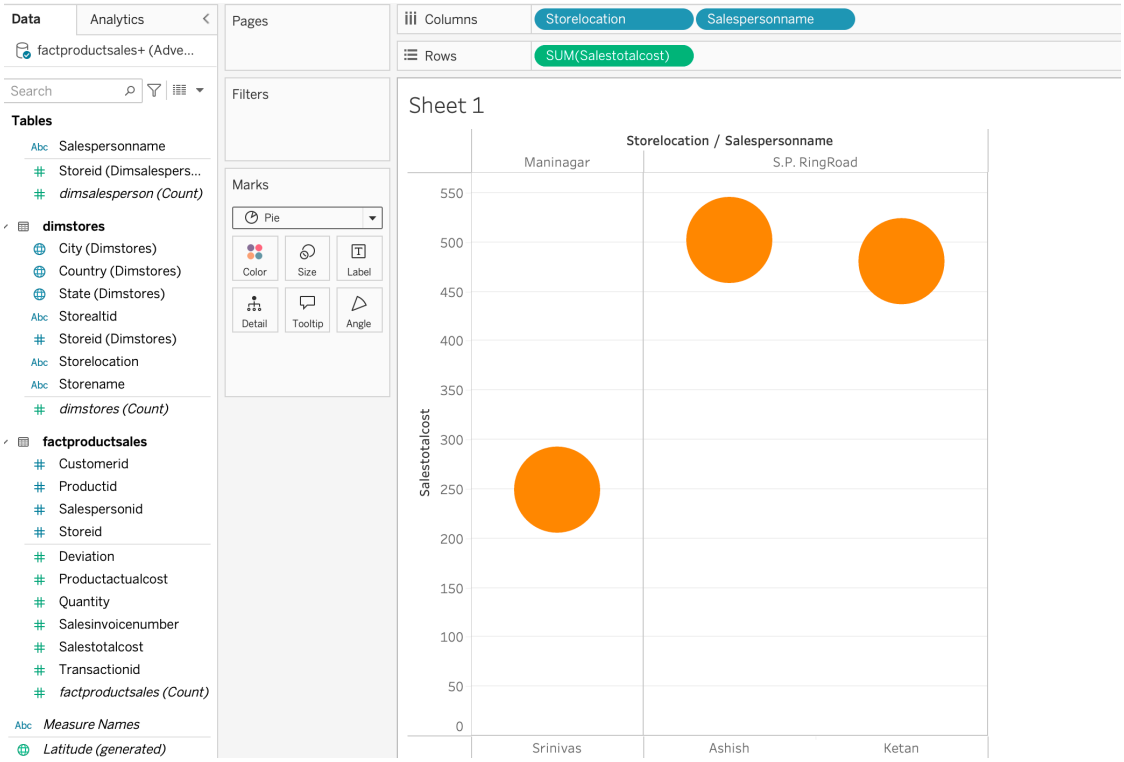
☑ Build a Dashboard:

- Drag and drop different visualization components from the "Sheets" onto the dashboard area.
- For example, to analyze sales performance:
  - **Sales by Store:** Use a bar chart to display total sales and quantity per store.



☑ Filter and Drill-Down Capabilities:

- Add filters for dimensions like `storename`, `productname`, `date`, etc., to allow interactive exploration of the data.
- Enable drill-down features by setting up hierarchies in the product and store dimensions, allowing more detailed analysis at different levels (e.g., city, state).



☑ Tool Tips and Info:

- Customize tool tips to show additional information when hovering over parts of the visualizations, such as exact sales figures, differences between periods, or details about the salesperson.

Step 4: Publish, Share, and Export

Once your dashboard is ready, you can share it with stakeholders in multiple ways:

- **Publish to Tableau Server or Tableau Online**:
  Go to the "Server" menu, select "Publish Workbook," choose the server you're connected to, configure the permissions, and publish your dashboard to share it with others.
- **Export as a PDF**:
  If you need to share your dashboard as a document, you can generate a PDF file. Go to the "File" menu, select "Print", and then select "Save as PDF". This is useful for presentations.

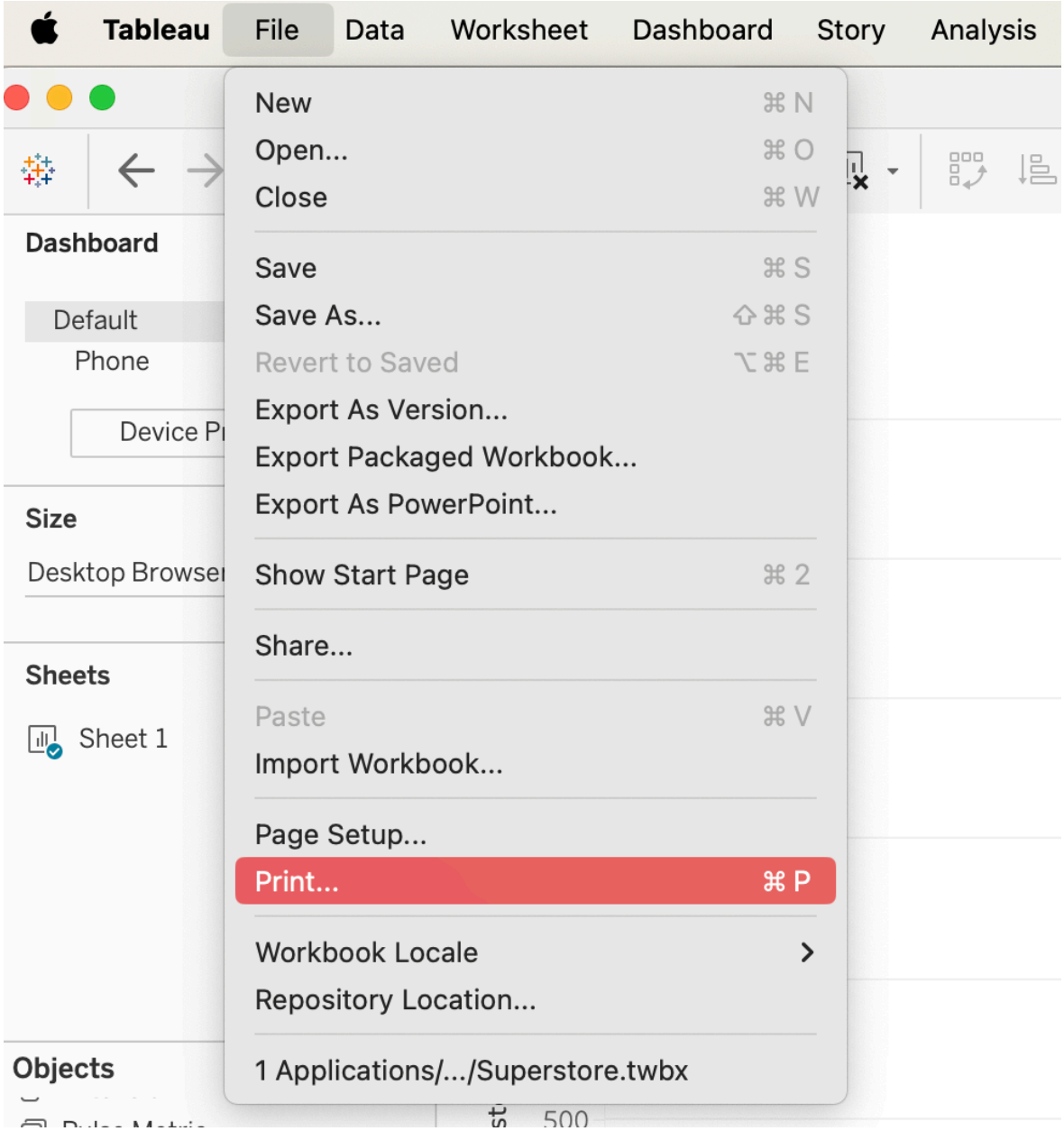
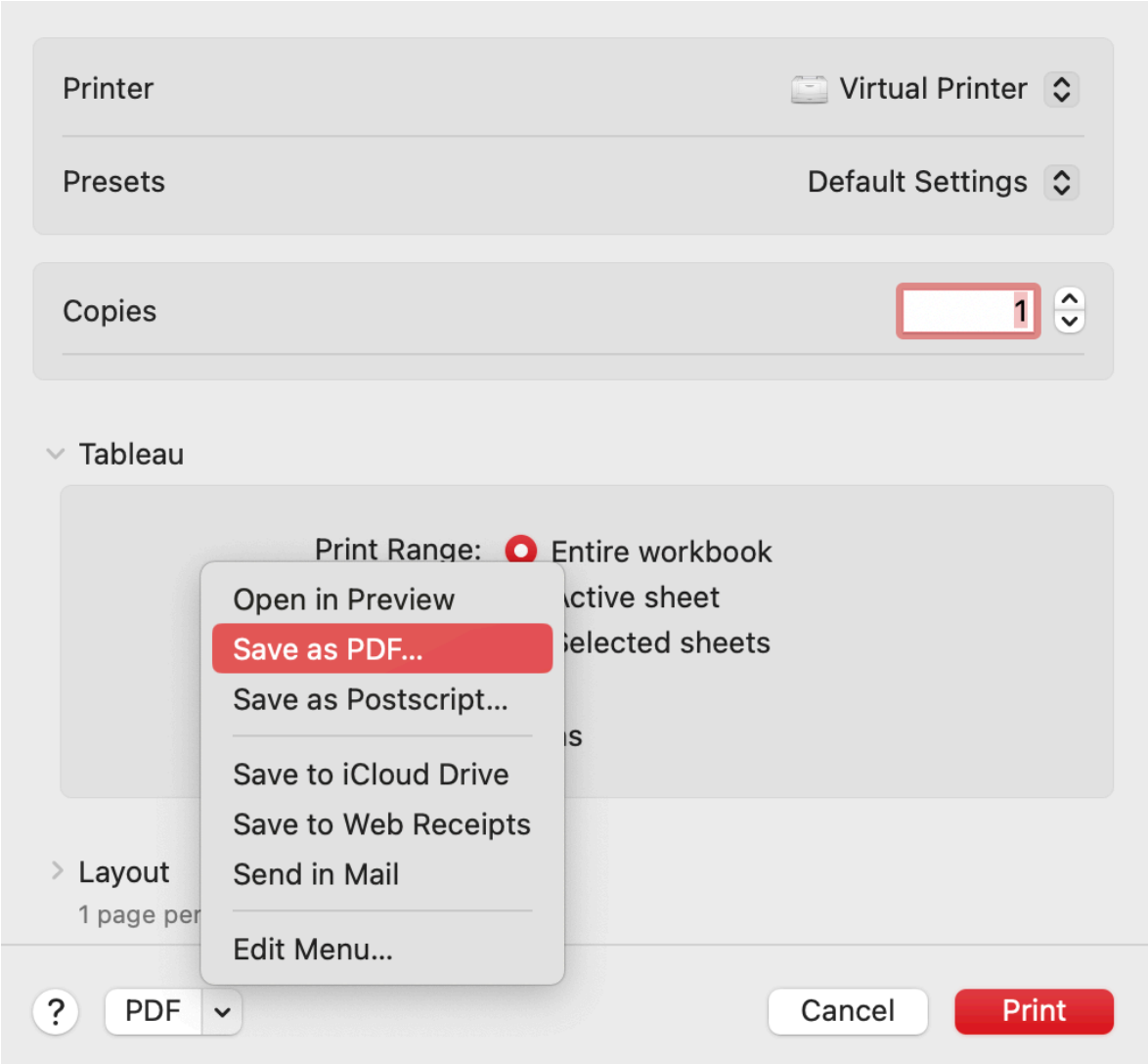


Tableau
File
Data
Worksheet
Dashboard
Story
Analysis
New ⌘ N
Open... ⌘ O
Close ⌘ W
Save ⌘ S
Save As... ⇧⌘ S
Revert to Saved ⌥⌘ E
Export As Version...
Export Packaged Workbook...
Export As PowerPoint...
Show Start Page ⌘ 2
Share...
Paste ⌘ V
Import Workbook...
Page Setup...
Print... ⌘ P
Workbook Locale
Repository Location...
1 Applications/.../Superstore.twbx
Dashboard
Default
Phone
Size
Sheets
Sheet 1
Objects



Please Note:

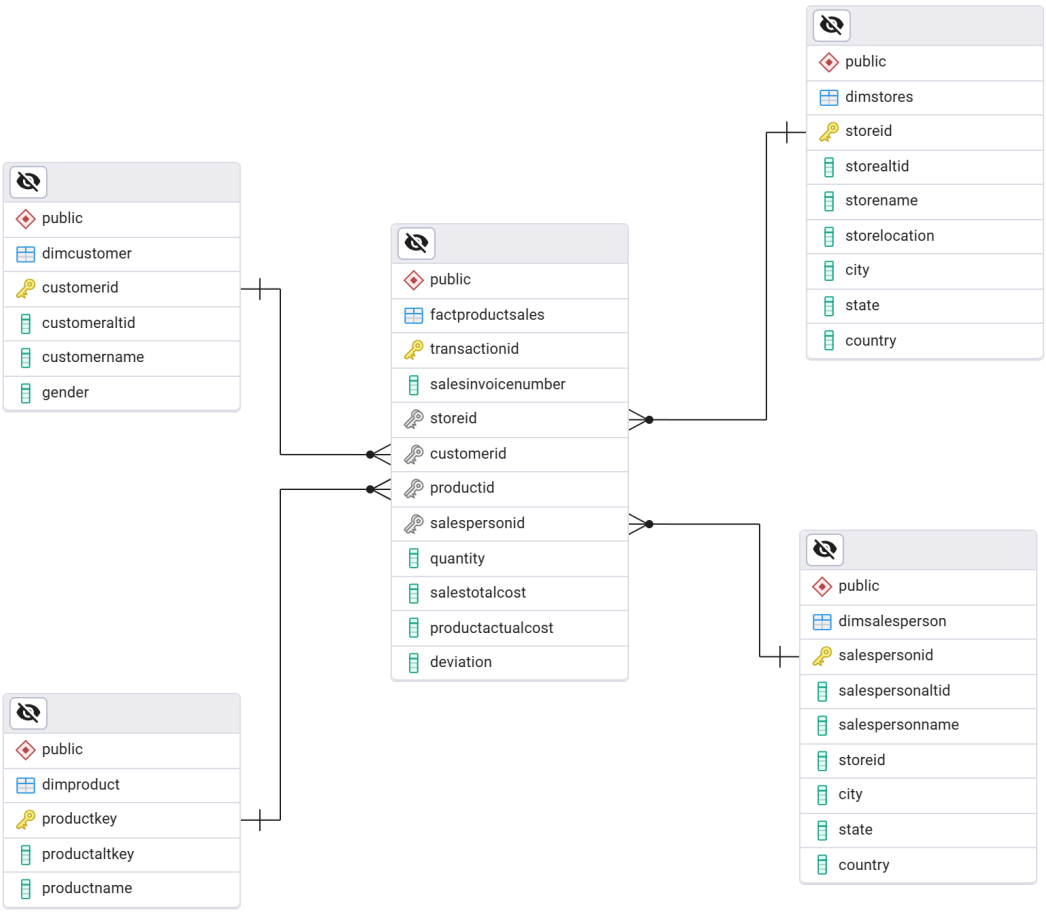
If the system shows that no printer is available, please add a printer first (virtual printer also works.

Part 6: Assessment

Demonstrate your ability to design and work with a star schema data model, query data for specific business questions, and visualize results using Tableau or Power BI. Ensure correct understanding and application of concepts learned during Weeks 1–5.

1. Create a database and populate it with sample data.

2. Reproduce the schema on your machine.



3. Draw a starnet with footprints to illustrate the business queries below:

- *What is the total sales cost of Nirma Soap in Maninagar?*
- *What is the total product actual cost of Arial Washing Powder 1kg sold by salesperson Jacob at X-Mart stores?*

4. Write SQL queries using `GROUP BY CUBE` to answer the business queries above.
5. Create visualisations to display the query results
6. Instead of the current design, Salesperson and Store could each be a separate dimension, with Location extracted into its own independent dimension. What are the advantages and disadvantages of this alternative approach compared to the existing design?

⚠

- Present your solutions to a lab facilitator during Week 5 or Week 6 lab sessions.
- Multiple submissions are permitted for feedback and improvement.
- Lab facilitators will provide guidance to help you correct any errors.

😄

Marking Rubric

Items	Completed (5)	Uncompleted (0)
Assessment tasks	6 tasks have been completed correctly.	Missing any 1 task.

Due date/time: 23:59 on 2 April, Thursday.

Submission Procedure: During weeks 5 and 6 labs.

⚠ **Submissions for lab demos through cssubmit, LMS, emails, and Teams will not be accepted.**

Previous
Week 4 - Designing a New Data Warehouse Using Databricks

Next
Week 6 - Association Rules Mining

Last updated 17 days ago